

Manual de Python: Estructuras y Funciones

Guía de referencia técnica — Tuplas, Listas, Diccionarios y Funciones con todos sus métodos

En la programación con Python, el dominio de las estructuras de datos fundamentales y la correcta modularización mediante funciones constituyen la base para construir aplicaciones sólidas, eficientes y mantenibles. Este manual reúne las definiciones esenciales, tablas comparativas y el compendio exhaustivo de métodos integrados.

1. Estructuras de Datos Universales

Python ofrece tres estructuras de datos primarias para la agrupación de elementos: **Listas**, **Tuplas** y **Diccionarios**. Cada una responde a necesidades específicas en términos de mutabilidad, orden e indexación.

Estructura	Mutable	Ordenada	Indexación	Sintaxis	Caso de uso principal
Lista (<code>list</code>)	Sí	Sí	Posicional (0, 1, ...)	<code>[a, b, c]</code>	Colecciones dinámicas donde los elementos cambian o se reordenan.
Tupla (<code>tuple</code>)	No	Sí	Posicional (0, 1, ...)	<code>(a, b, c)</code>	Datos inmutables protegidos contra modificaciones inadvertidas.
Diccionario (<code>dict</code>)	Sí	Sí*	Por Clave (Key)	<code>{"k": v}</code>	Mapeo de atributos, registros estructurados o relaciones clave-valor.

Nota técnica: Desde Python 3.7+, los diccionarios garantizan el mantenimiento del orden de inserción de las claves. Sin embargo, su acceso principal sigue realizándose mediante la clave y no mediante un índice numérico.

2. Listas (`list`)

Las listas son secuencias mutables diseñadas para almacenar colecciones homogéneas o heterogéneas. Permiten la inserción, eliminación y reordenamiento directo de sus elementos.

2.1. Todos los Métodos de Listas

A continuación se presenta el uso práctico de los 11 métodos propios de las listas dentro de una función de gestión de inventario:

```
def demostrar_metodos_listas():
    # Inicialización
    productos = ["laptop", "mouse"]

    # 1. append(x): agrega al final
    productos.append("teclado")          # ['laptop', 'mouse', 'teclado']

    # 2. insert(i, x): inserta en posición i
    productos.insert(1, "monitor")       # ['laptop', 'monitor', 'mouse', 'teclado']

    # 3. extend(iterable): añade múltiples elementos
    productos.extend(["pantalla", "mouse"]) # Añade dos elementos

    # 4. count(x): cuenta las apariciones del elemento
    cant_mouses = productos.count("mouse") # Devuelve: 2

    # 5. index(x): devuelve el primer índice del elemento
    pos_monitor = productos.index("monitor") # Devuelve: 1

    # 6. sort(): ordena la lista en el lugar
    productos.sort()                    # Orden alfabético

    # 7. reverse(): invierte el orden actual
    productos.reverse()                 # Invierte lista

    # 8. copy(): crea una copia independiente
    copia_respaldo = productos.copy()

    # 9. pop([i]): extrae y devuelve el elemento en i (por defecto el último)
    ultimo_item = productos.pop()       # Devuelve y remueve el último

    # 10. remove(x): elimina la primera ocurrencia de x
    productos.remove("mouse")           # Remueve primer "mouse"

    # 11. clear(): vacía todos los elementos de la lista
    productos.clear()                   # Resultado: []

    return copia_respaldo
```

3. Tuplas (tuple)

Las tuplas son secuencias inmutables. Una vez creadas, su contenido no puede modificarse, añadirse ni eliminarse. Se utilizan para proteger la integridad de los datos.

3.1. Todos los Métodos de Tuplas

Dado su carácter inmutable, las tuplas poseen únicamente 2 métodos integrados:

```
def analizar_tupla_mediciones(datos):
```

4. Diccionarios (`dict`)

Los diccionarios almacenan asociaciones de clave-valor. Permiten la recuperación eficiente de información mediante identificadores únicos llamados claves.

4.1. Todos los Métodos de Diccionarios

Demostración de los 10 métodos integrados en diccionarios para la administración de perfiles:

```
def demostrar_metodos_diccionarios():
    perfil = {"id": 101, "usuario": "mgarcia", "rol": "admin"}

    # 1. get(key, default): lectura segura sin lanzar KeyError
    email = perfil.get("email", "sin_correo@dominio.com")

    # 2. setdefault(key, default): retorna valor o lo asigna si no existe
    estado = perfil.setdefault("activo", True)

    # 3. update(dict2): actualiza y añade pares clave-valor
    perfil.update({"rol": "superadmin", "ciudad": "Bogotá"})

    # 4. keys(): obtiene vista de todas las claves
    lista_claves = list(perfil.keys())

    # 5. values(): obtiene vista de todos los valores
    lista_valores = list(perfil.values())

    # 6. items(): obtiene vista de pares (clave, valor)
    lista_pares = list(perfil.items())

    # 7. copy(): crea una copia superficial
    respaldo = perfil.copy()

    # 8. pop(key): elimina la clave y devuelve su valor
    rol_eliminado = perfil.pop("rol")

    # 9. popitem(): elimina y devuelve el último par (clave, valor)
    ultimo_par = perfil.popitem()

    # 10. clear(): elimina todas las parejas clave-valor
    perfil.clear()

    return respaldo
```

5. Funciones en Python

Las funciones son bloques de código reutilizables delimitados por la palabra clave `def`. Facilitan la abstracción, el mantenimiento modular y previenen la duplicación de lógica.

- **Parámetros:** Variables de entrada recibidas por la función.
- **Valores por defecto:** Parámetros opcionales con valores predefinidos.
- **Retorno (`return`):** Transfiere un resultado formal al contexto donde fue invocada.

5.1. Ejemplos Avanzados de Funciones

A continuación se ilustran dos patrones esenciales: uso de parámetros por defecto e integración total con las estructuras analizadas.

A. Parámetros Nombrados y Valores Predeterminados

```
def calcular_factura(monto_base, impuesto=0.19, descuento=0.0):  
    """  
    Calcula el costo total aplicando un descuento previo e impuestos.  
    """  
    monto_descontado = monto_base * (1 - descuento)  
    monto_total = monto_descontado * (1 + impuesto)  
    return round(monto_total, 2)  
  
# Llamadas con distintas configuraciones de argumentos  
total_estandar = calcular_factura(100.0) # Impuesto 19%, desc 0% -> 119.0  
total_promocion = calcular_factura(100.0, descuento=0.10) # Impuesto 19%, desc 10% -> 107.1
```

B. Integración de Funciones con Estructuras Combinadas

```
def procesar_evaluaciones(estudiantes):  
    """  
    Recibe una lista de diccionarios y retorna una tupla estructurada:  
    (promedio_general, mejor_estudiante)  
    """  
    if not estudiantes:  
        return (0.0, None)  
  
    suma_notas = 0.0  
    mejor_estudiante = estudiantes[0]  
  
    for est in estudiantes:  
        suma_notas += est.get("nota", 0.0)  
        if est.get("nota", 0.0) > mejor_estudiante.get("nota", 0.0):  
            mejor_estudiante = est  
  
    promedio = suma_notas / len(estudiantes)  
    return (round(promedio, 2), mejor_estudiante.get("nombre"))  
  
# Datos de entrada  
grupo = [  
    {"nombre": "Ana", "nota": 4.5},  
    {"nombre": "Carlos", "nota": 3.8},  
    {"nombre": "Elena", "nota": 4.9}  
]  
  
prom, mejor = procesar_evaluaciones(grupo)  
# Resultado: prom = 4.4, mejor = 'Elena'
```